

agent-meow 实施范围报告

计划 001-010 · 完整实施路线图 (橘宝R16)

运行环境: 橘宝R16 · AMD Ryzen AI 9 HX 470 + RTX 5060 (8GB CUDA)

四引擎: CPU 12C + dGPU 8GB CUDA + iGPU 890M + NPU XDNA 2 · **日期:** 2026-08-04

状态: 可行性评估中 · 灵创K16 方案已推送至 `origin/main`

硬件平台 & HX470+5060 架构

组件	规格	状态
CPU	AMD Ryzen AI 9 HX 470, 12C/24T (Gorgon Point)	✓
iGPU	Radeon 890M (RDNA 3.5)	✓
iGPU 显存	32GB 统一内存	✓
dGPU	RTX 5060 Laptop (8GB GDDR7, Blackwell)	✓
dGPU 计算	CUDA + Vulkan (原生支持)	✓
NPU	AMD XDNA 2 (~50 TOPS)	✓
CUDA	RTX 5060 原生 CUDA 支持	✓
ROCm	7.1 (iGPU Vulkan 优先)	✓
内存	32GB DDR5-5600	✓

HX470+5060 独特性： RTX 5060 提供原生 CUDA → faster-whisper **直接可用**，无需 whisper.cpp Vulkan 替代。8GB GDDR7 独立显存 + 32GB 统一内存 = 混合 offload 架构。预填充 A3B 活跃层 + Whisper 到 dGPU 后，iGPU 890M + NPU 通过统一内存承担辅助计算。

计划 001-005：基础设施修复

#	计划	状态	内容
001	db_models 路径	TODO	路径精确化
002	VIDEOS_SURFACE.md	TODO	过时声明
003	Phase 4 runner	TODO	注册工具
004	过时 voicebox	TODO	清理废弃
005	Voicebox 可靠性	DRAFT	被 006+008 取代

优先级：低于 QAA 语音迁移。代码卫生修复不依赖硬件平台，与灵创K16 完全通用。

计划 006+006b: QAA 网关 + 离线预热架构

已解决: 原 S2S 冷启动 90 秒问题已通过 GPU STT + 开机预热机制解决。

橘宝R16 优势: RTX 5060 有 **CUDA** → faster-whisper 直接可用，无需替代方案！

离线方案: QAA 网关本地模式 + faster-whisper/Ollama/Kokoro 开机后台预热，用户无感启动。

指标	优化前	已实现
预热	90s	~3s 离线（或 ~0s 热池）
成本	免费	零（完全本地）

浏览器 → Vite(ws:true) → QAA 网关 (:3101)
├ STT: faster-whisper CUDA (GPU, ~1s 预热)
├ LLM: Ollama CUDA (GPU, ~3-5s 预热)
└ TTS: Kokoro (CPU, ~0s 预热)

风险: LOW · 工作量: S · 开机预热机制是架构级能力（混合在线/离线可后续扩展）

计划 007：QAA 语音钩子 → MeowCat 界面

保留猫爪 UI，替换传输层 — 与硬件无关

旧组件	新组件	动作
realtimeVoice.ts (221行)	QAA useRealtimeVoice.js	替换
s2s_proxy.py (233行)	QAA 网关	替换
猫爪按钮+波形	保留	不变

协议差异：QAA 用 `GatewayClientEvent` JSON 协议 vs 当前自定义二进制帧。需重写事件处理器，React 组件树不变。风险: MED · 工作量: L

计划 008: faster-whisper + CUDA GPU STT

已解决: 橘宝R16 的 RTX 5060 有 **CUDA** → faster-whisper 直接可用，无需 whisper.cpp 替代。

方案: 直接安装 faster-whisper，STT 放到 RTX 5060 dGPU。 **无需编译 whisper.cpp**。

组件	优化前	已实现	位置
STT	CPU 60s	GPU ~1s	dGPU
TTS	CPU 30s	~0s 预热	CPU
总预热	90s	~3s 或 0s	—

显存: whisper-large-v3 ~1.5GB 在 8GB GDDR7，充裕。 风险: **LOW** • 工作量: **S** (`pip install faster-whisper`)

对比灵创K16: K16 需编译 whisper.cpp + Vulkan (MED 难度); R16 仅 `pip install` (LOW 难度)

计划 009+010： ACP 垫片 → Hermes → Ollama 本地

009： ACP 垫片解耦语音前端与 Hermes 代理 OS。简单问题即时回答；工具调用 `spawn_thinking` 后台异步执行，完成后播报。

010： Ollama + CUDA (RTX 5060) 本地 GPU 推理。

指标	Hermes Docker	Ollama 本地 (CUDA)
推理	远程 API	GPU (CUDA)
延迟	~1.8ms	~0.3ms
成本	API 费用	零

模型策略： Qwen3.6-35B-A3B (MoE, 仅 3B 激活) — 与灵创K16 同模型架构

- 活跃层 (3B) 驻留 8GB dGPU GDDR7
- 256 专家分布 dGPU + iGPU 890M + 系统 RAM (IQ3_XXS ~13GB 或 Q4_K_M ~22GB)
- iGPU 890M 通过 32GB 统一内存辅助 MoE 专家推理
- NPU XDNA 2 (~50 TOPS) 承担辅助推理/未来 STT 卸载

HX470+5060 四引擎混合 offload 优化栈

组件	引擎	位置	预热	计划
LLM (35B-A3B MoE)	Ollama+CUDA	dGPU 8GB+RAM+iGPU	~3-5s	010
STT (Whisper)	faster-whisper+CUDA	dGPU	~1s	008
TTS (Kokoro)	Kokoro-82M	CPU	~0s	008
VAD (Silero)	Silero	CPU	~0s	现有
语音网关	QAA (Node.js)	CPU	~2s	006
代理 OS	Hermes/Ollama	CPU+dGPU	已运行	009/010
MoE 专家 offload	iGPU 890M+RAM	iGPU 32GB 统一内存	已预填充	010
辅助推理	NPU XDNA 2	NPU ~50 TOPS	已就绪	010
前端	React+Vite	浏览器	即时	007

显存预算：8GB GDDR7 = MoE 活跃层 3B (~3GB) + STT 1.5GB = 4.5GB，剩余 3.5GB。

统一内存：32GB = MoE 专家层 (IQ3_XXS ~13GB) + 系统开销。预填充后 iGPU 890M + NPU 均活跃。

依赖关系与执行顺序

006 (QAA 网关) → 007 (语音钩子) ← 008 (faster-whisper CUDA)
006 → 009 (ACP 垫片) → 010 (Ollama LLM CUDA)
006b (预热接线) → 007

阶段	计划	说明
1	006 + 008	并行 (008 = pip install)
2	006b + 007	依赖 006
3	009	依赖 006
4	010	依赖 009

对比灵创K16：橘宝R16 的计划 008 更简单（`pip install faster-whisper`，无需编译 whisper.cpp）。计划 010 使用同一 35B-A3B MoE 模型，但 IQ3 量化 + GPU/RAM 混合 offload。

已实现效果

指标	优化前	已实现
语音预热	90s	~3s 离线 / ~0s 热池
LLM 推理	远程 API	本地 GPU (CUDA, 8GB+RAM)
STT 推理	CPU 60s	GPU CUDA ~1s
云端依赖	必须	零 (完全离线)
成本	API 费用	零 (离线)
GPU 利用率	0%	四引擎全活跃
代理能力	无	Hermes OS

agent-meow 在橘宝R16：四引擎协同的本地 AI 语音代理。dGPU（Ollama + CUDA）跑 MoE 活跃层 + STT，iGPU 890M 通过 32GB 统一内存辅助 MoE 专家推理，NPU XDNA 2 承担辅助计算，CPU 跑 TTS + 网关 + 代理 OS。预填充 A3B + Whisper 后所有引擎活跃，零云端，零外部 GPU。**实现难度低于灵创K16**（CUDA 原生 vs Vulkan 编译）。